SIT151 - Assessment 4

Game Development & Implementation

Student Name:	Daniel Mattioli
Student Number:	S223377562
Unit:	SIT151 - Game Fundamentals
Submission Date:	Due: Friday, Week 12, 8pm AEST

Chosen Features Summary

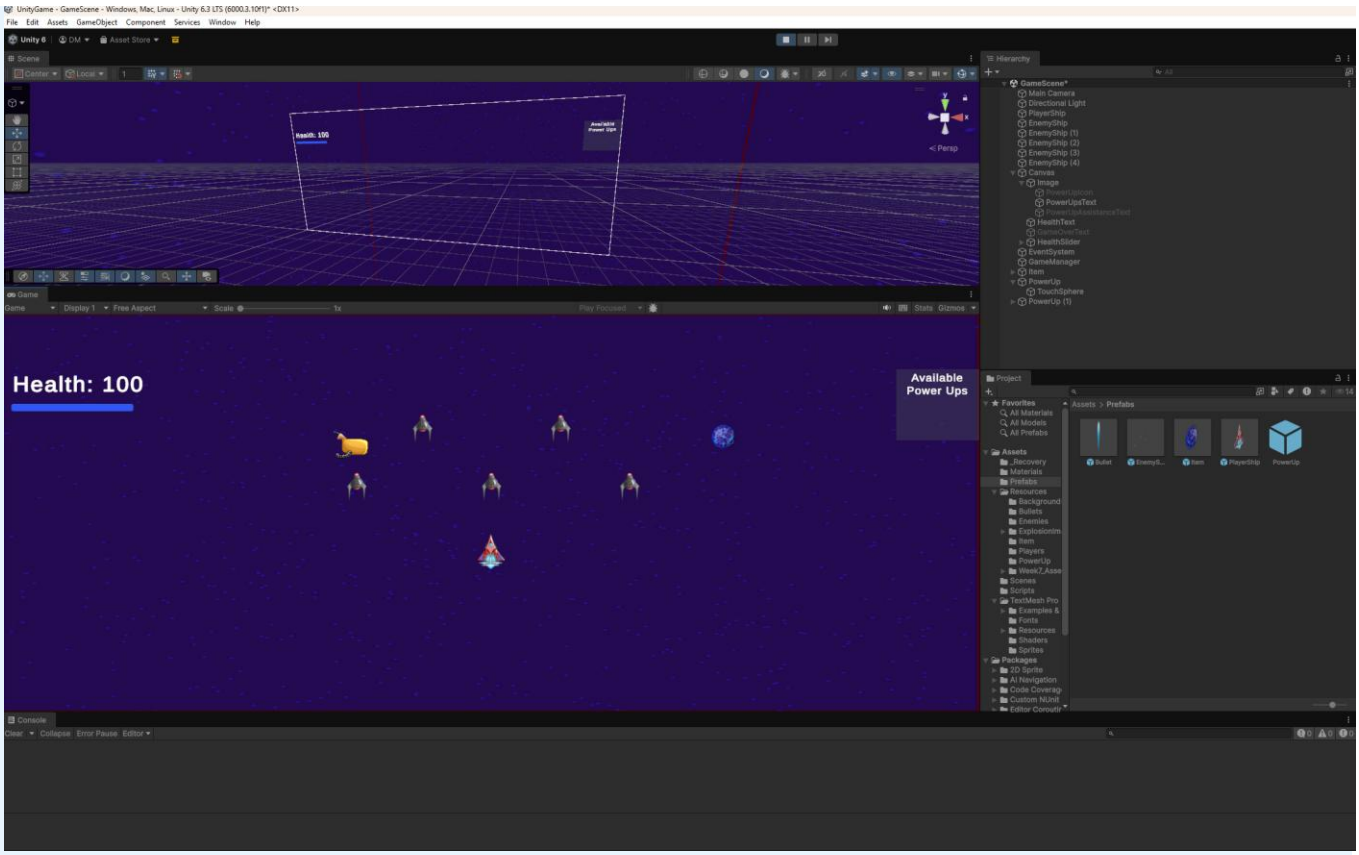
#	Component	Feature	Feature No.
1	Power-Up	Collectible power-up (stored/activated by player)	Feature 1
2	Power-Up	Power-up enhances player abilities in interesting way	Feature 2
3	Victory Condition	Game ends when player achieves creative win state	Feature 1
4	Victory Condition	Real-time feedback showing progress to win state	Feature 2

Video Demo Link: https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=525004bb-8b89-43fa-b56b-b45500cd9a3a

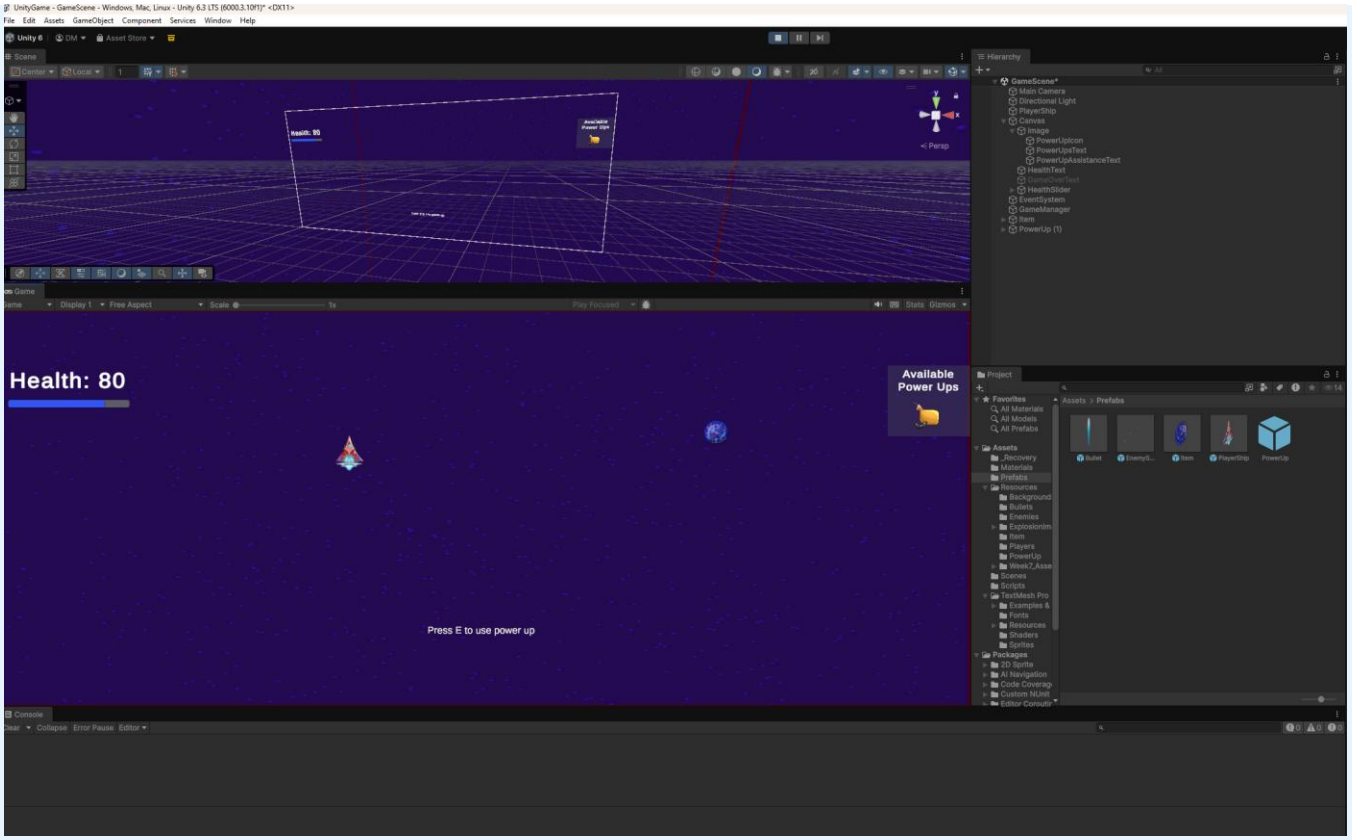
Feature 1: Power-Up - Feature 1: Collectible Power-Up

DESCRIPTION

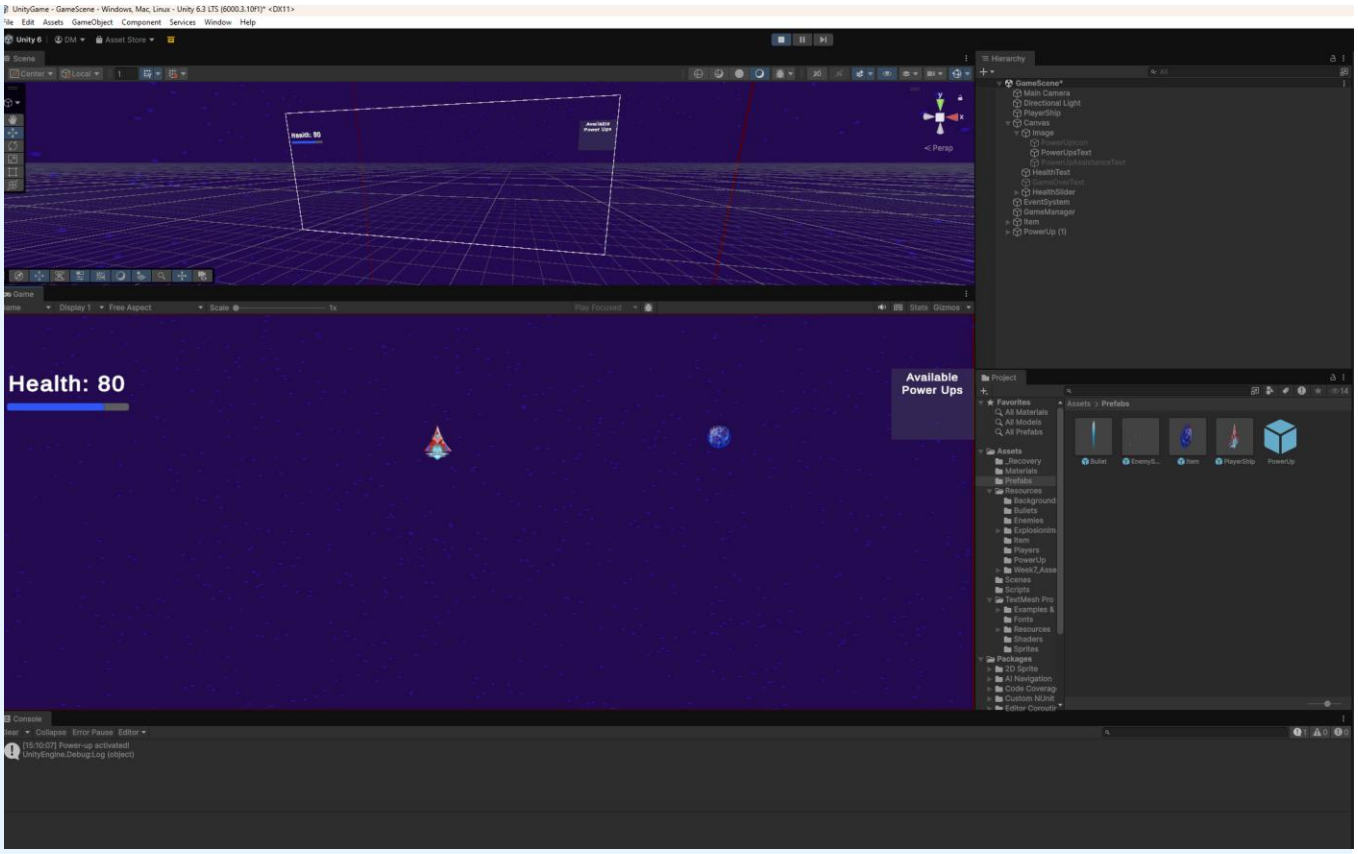
A collectible power-up object (a fuel canister) is placed in the game world and can be collected by the player's rocket ship. Rather than activating immediately on pickup, the power-up is stored and can be activated by the player at a chosen moment by pressing a designated key (e.g., E). A visual indicator on the HUD shows whether a power-up is currently held.



At this point in the game, enemy ships are spawning and moving downward. The fuel canister power-up sprite is visible in the scene. The "Available Power Ups" HUD panel in the top-right is empty, indicating the player has not yet collected the power-up.



The player has flown over the fuel canister and collected it. The canister icon now appears in the "Available Power Ups" HUD panel in the top-right corner. A "Press E to use power up" prompt is displayed at the bottom of the screen, reminding the player they are holding a power-up and how to activate it.



After the player presses E, the power-up is consumed. The canister icon is removed from the HUD panel, and the "Press E" prompt disappears. The console confirms the activation with a "Power-up activated!" debug message. For the submission of this assessment and a final version of the game, the debug message has been removed from the code. The actual boost effect will be applied here in Feature 2.

REASONING

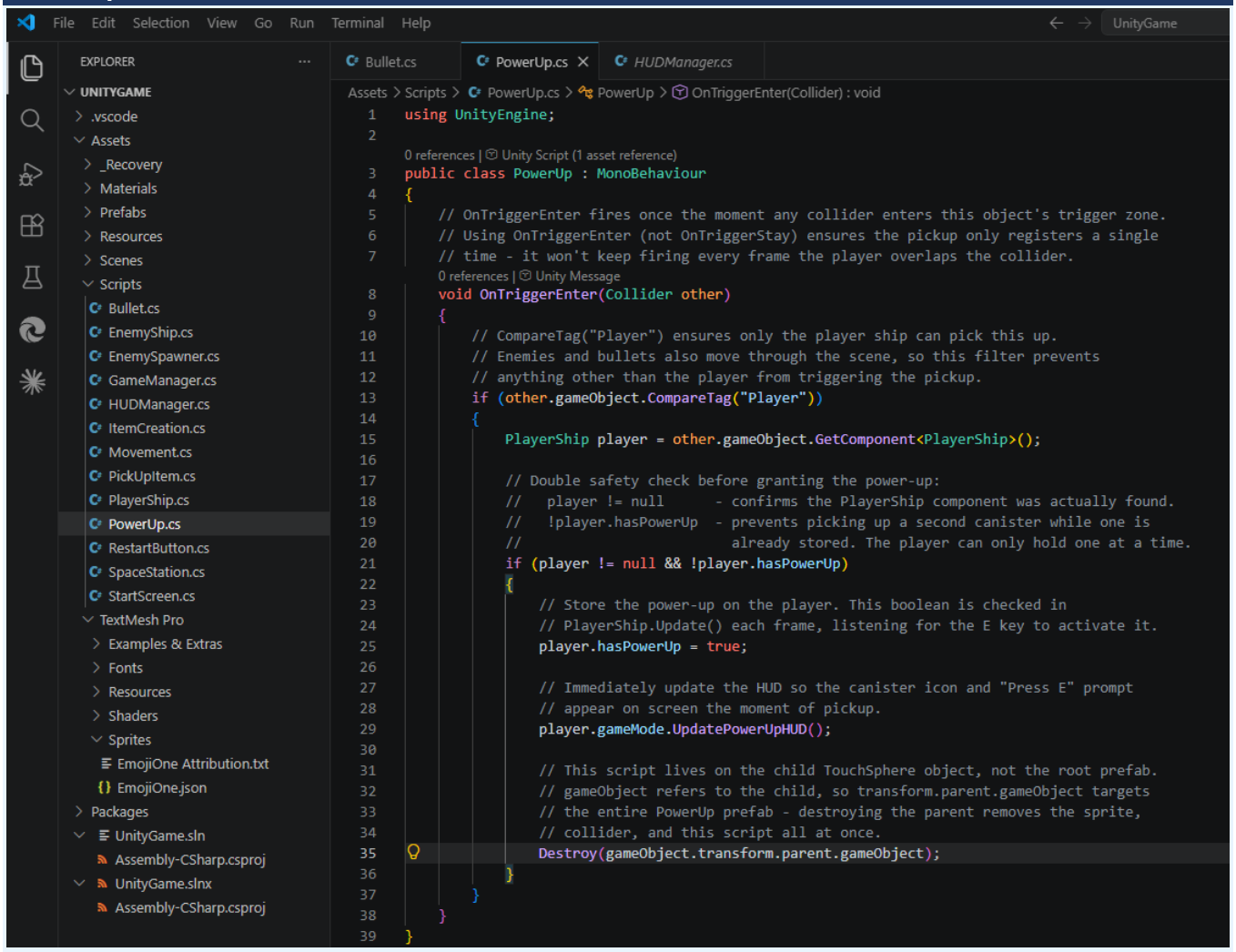
Storing a power-up rather than instantly applying it was a deliberate design choice to add player agency and strategic depth. In a fast-paced rocket ship game, giving the player the ability to choose when to use a power-up means they need to make real-time decisions - do I use it now, or save it for a tougher section? This creates more interesting and varied gameplay compared to an instant pickup effect. The fuel canister theme also fits naturally with a rocket ship game, keeping the game world internally consistent.

BLUEPRINTS/SCRIPTS AND IMPLEMENTATION

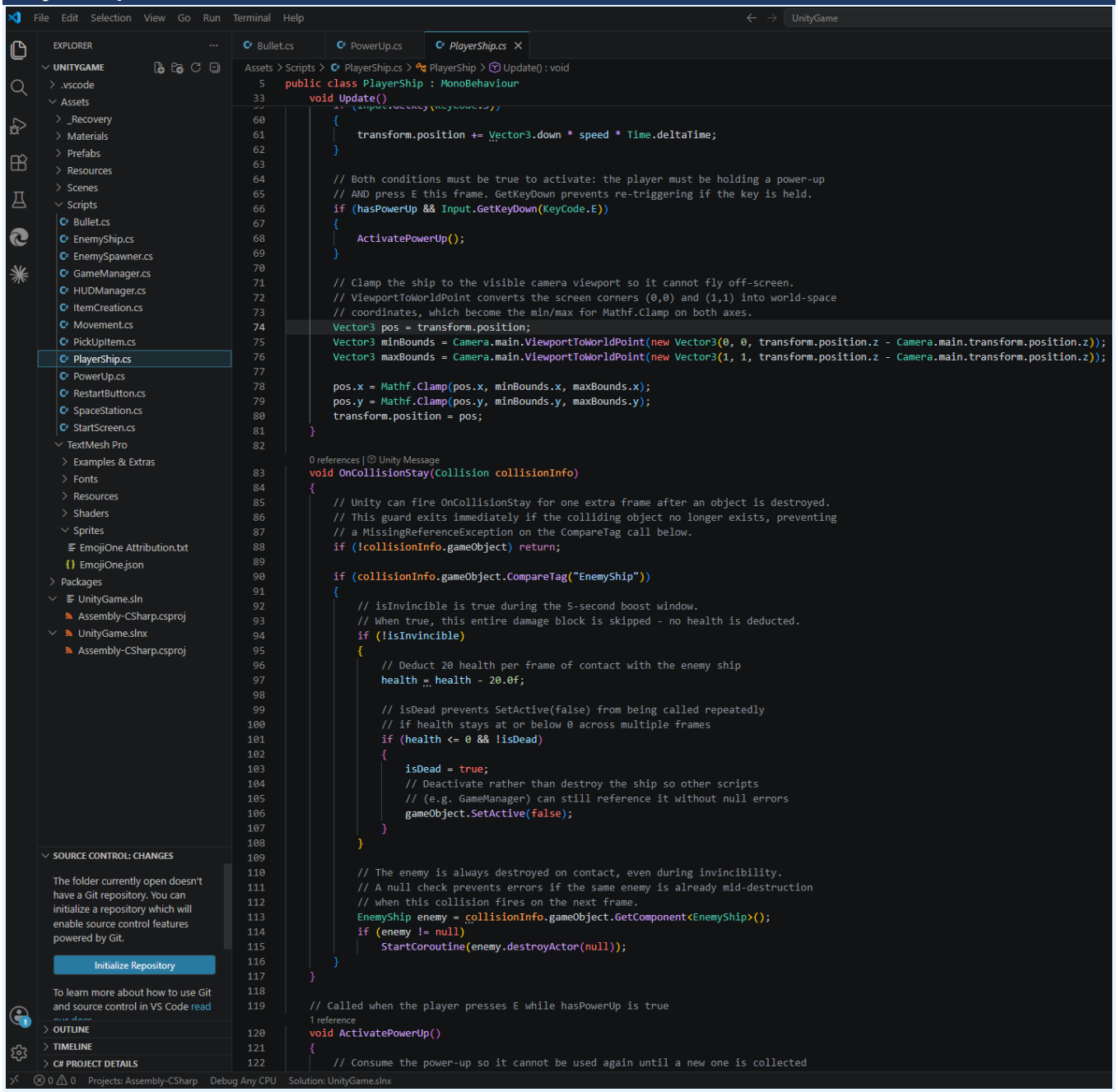
A **PowerUp** prefab was created with a parent object holding a Sprite Renderer and a child **TouchSphere** object containing a **Sphere Collider** set to 'Is Trigger'. The **PowerUp.cs** script on the TouchSphere uses OnTriggerEnter - not the 2D variant - as the game uses 3D physics. When the player's collider enters the trigger zone, the script checks CompareTag("Player") and verifies the player does not already hold a power-up before setting hasPowerUp = true on the **PlayerShip** component and destroying the parent object. In **PlayerShip.cs**, the Update() method checks for Input.GetKeyDown(KeyCode.E) to activate the stored power-up. In **GameManager.cs**, the

UpdatePowerUpHUD() method uses SetActive() to toggle the canister icon and "Press E" prompt on the Canvas based on the current value of hasPowerUp.

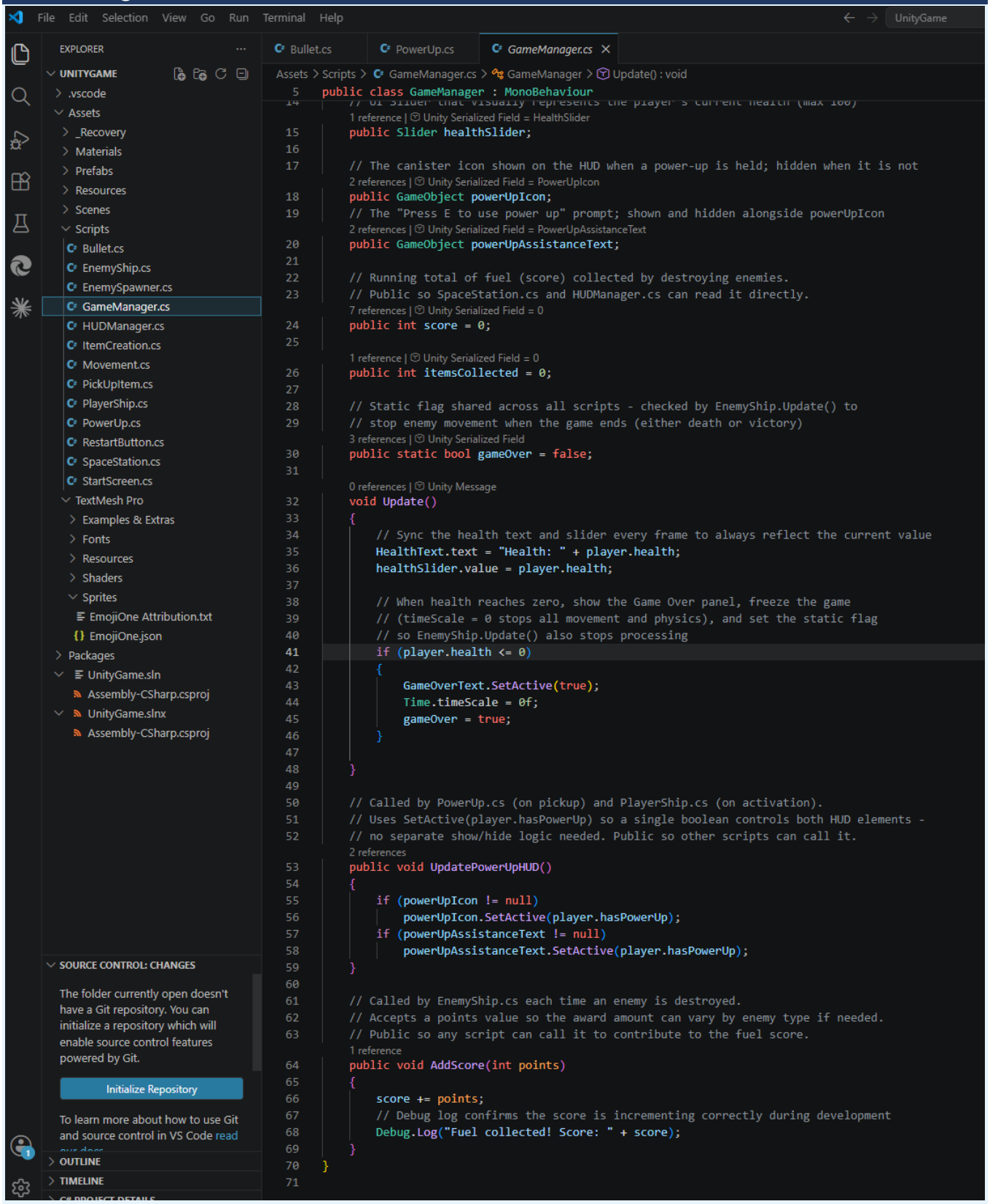
PowerUp.cs



PlayerShip.cs



GameManager.cs

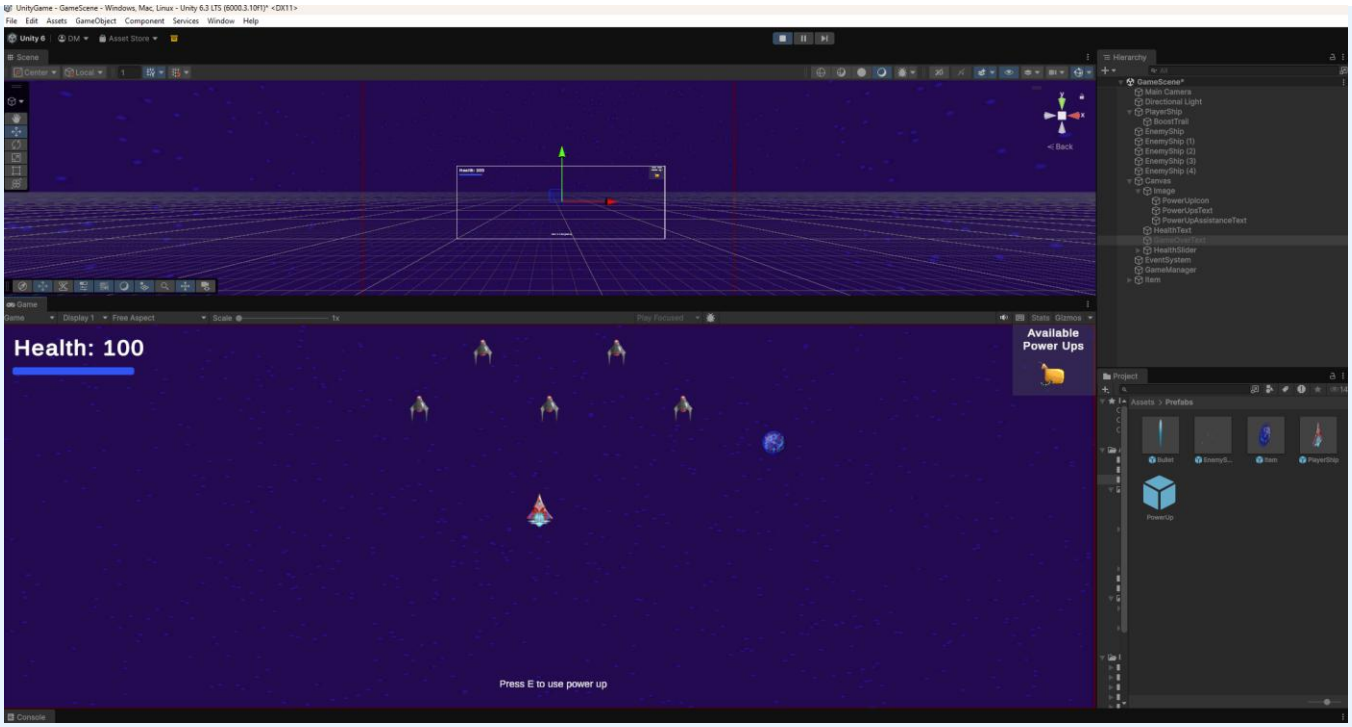


Please see the inline comments in PowerUp.cs, PlayerShip.cs and GameManager.cs for a full explanation of the pickup logic and hasPowerUp flag.

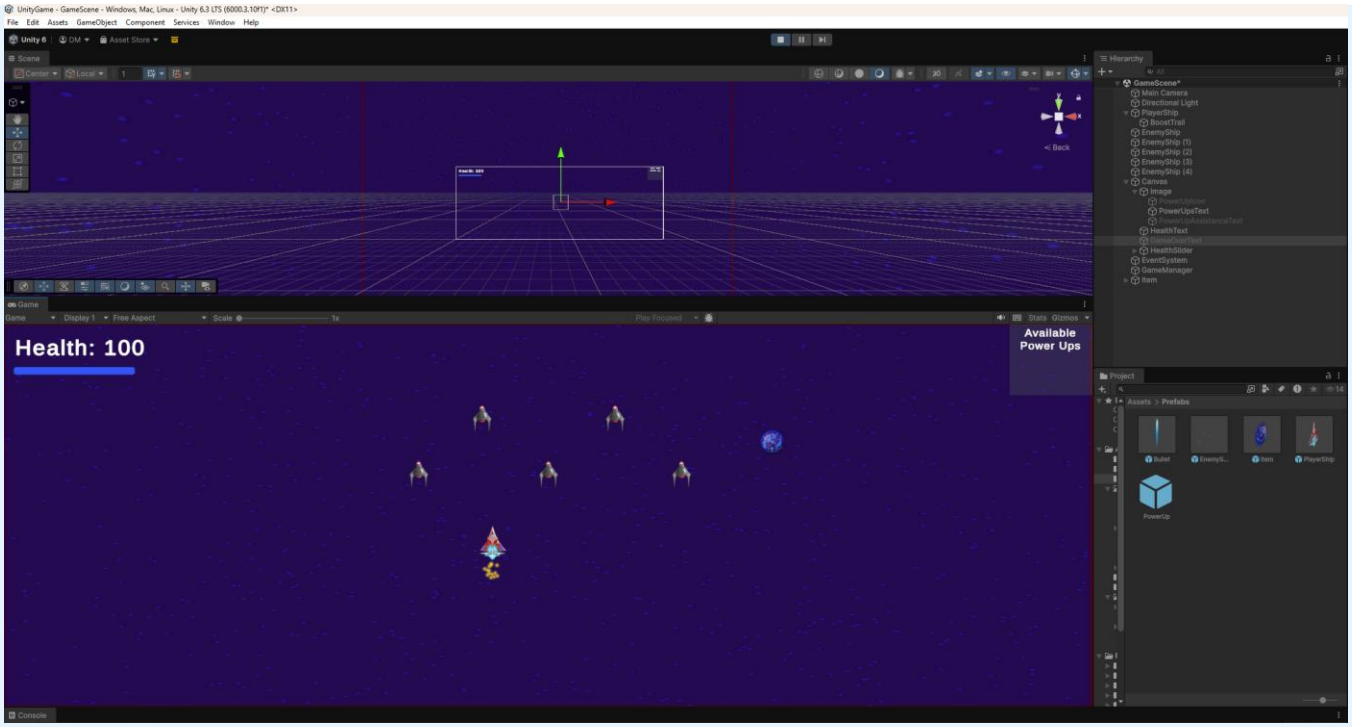
Feature 2: Power-Up - Feature 2: Power-Up Enhances Player Abilities

DESCRIPTION

When the stored fuel canister power-up is activated, the player's rocket enters a 'Boost Mode' for 5 seconds: movement speed increases by 50%, the ship becomes invincible to enemy contact damage (visualised by the ship flashing), and a particle trail effect activates behind the ship. After the duration, the ship returns to normal stats and the HUD indicator resets.



The fuel canister power-up has been collected. The canister icon is visible in the "Available Power Ups" HUD panel in the top-right corner and the "Press E to use power up" prompt is displayed at the bottom. The boost has not yet been activated and the ship is moving at normal speed and is not invincible. This screenshot demonstrates the stored state of the power-up before the player chooses to use it.



The player has pressed E, triggering the 5-second Boost Mode. The particle trail is visible beneath the ship, indicating the boost is active. During this state the ship's movement speed is increased by 50%, the ship is invincible to enemy contact damage, and the SpriteRenderer flashes on and off every 0.1 seconds to signal the invincibility to the player. After 5 seconds all three effects are automatically removed and the ship returns to its normal state.

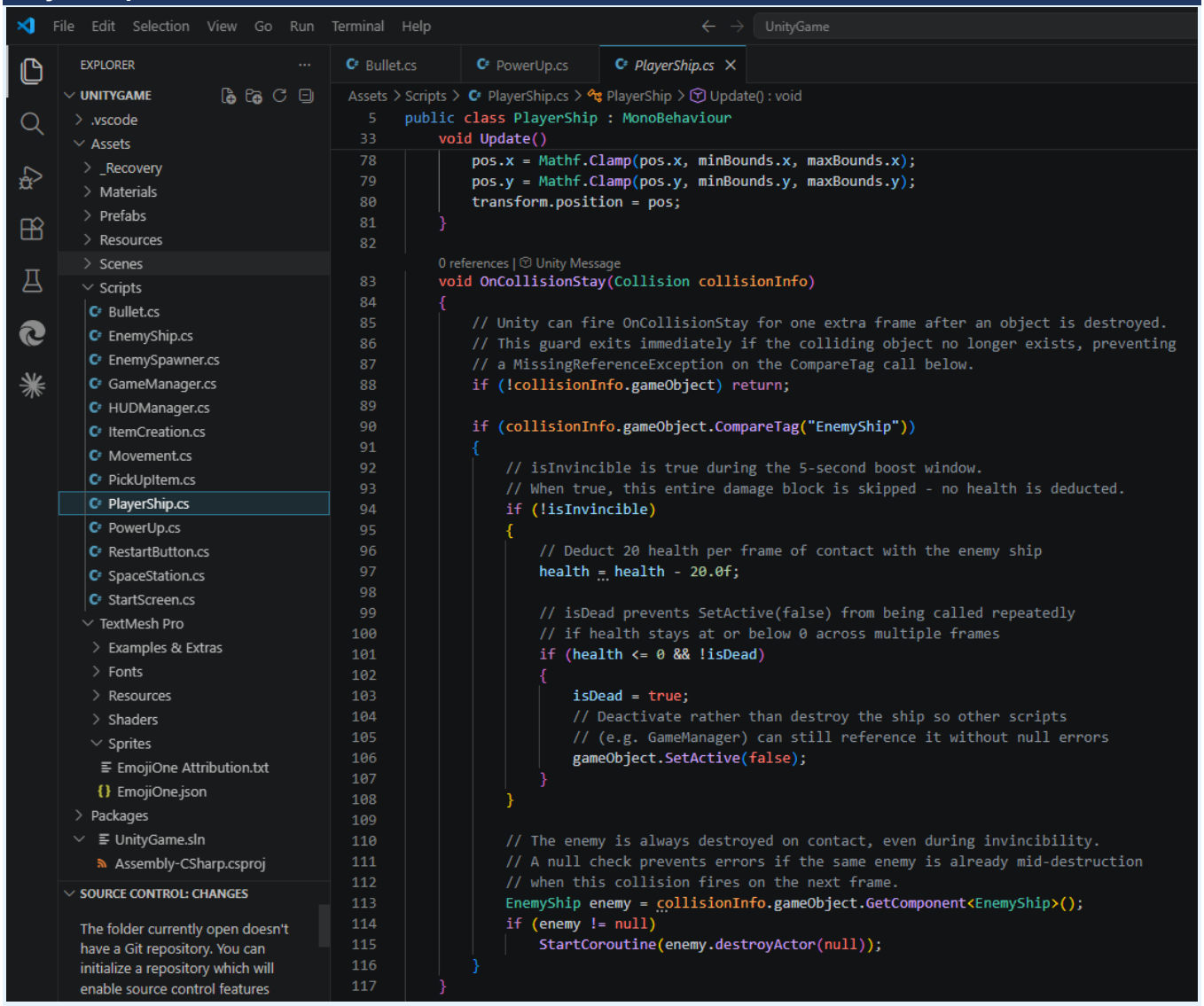
REASONING

Rather than a simple health restore, a timed speed boost with brief invincibility adds meaningful gameplay variety. It rewards skilled players who save their power-up for difficult moments (e.g., when surrounded by enemies or on low health). The combination of speed, invincibility, and a particle trail makes the power-up visually distinctive and satisfying to use - the player has a clear 'power fantasy' moment. The 5-second duration was chosen to be impactful but not trivially long.

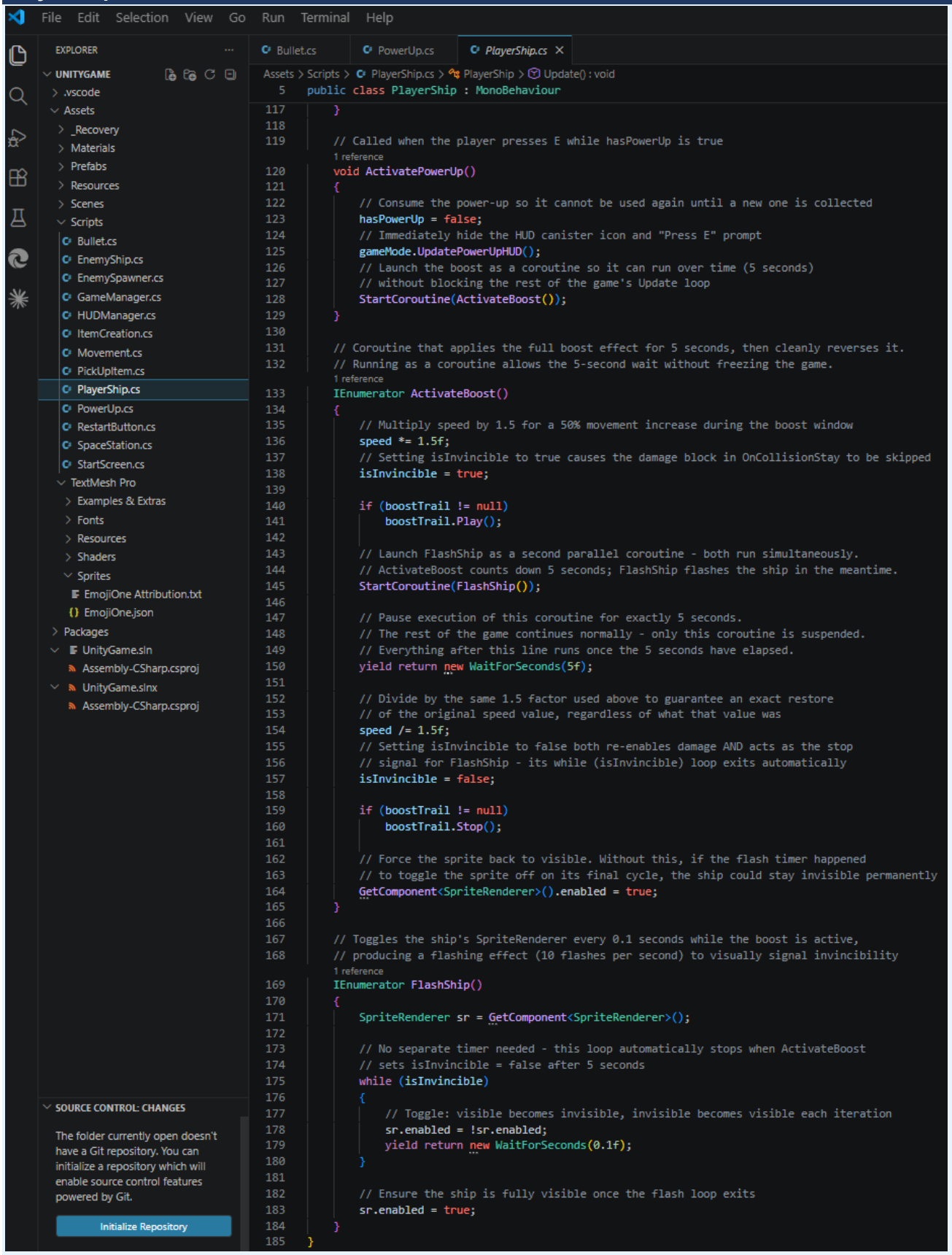
BLUEPRINTS/SCRIPTS AND IMPLEMENTATION

Feature 2 was implemented entirely within PlayerShip.cs. When the player presses E and hasPowerUp is true, the ActivatePowerUp() method is called, which consumes the power-up, updates the HUD, and launches the ActivateBoost() coroutine. Inside ActivateBoost(), the ship's speed is multiplied by 1.5 and the isInvincible boolean is set to true, which blocks the damage code inside OnCollisionStay from running. The method then calls boostTrail.Play() to start the particle trail effect, and simultaneously launches a second coroutine, FlashShip(), which toggles the SpriteRenderer on and off every 0.1 seconds to create a flashing visual. After a WaitForSeconds(5f) delay, the boost effects are reversed - speed is divided back to its original value, isInvincible is set to false (which also causes FlashShip() to exit its while loop), the particle trail is stopped, and the SpriteRenderer is forced back to enabled to ensure the ship is fully visible.

PlayerShip.cs



PlayerShip.cs

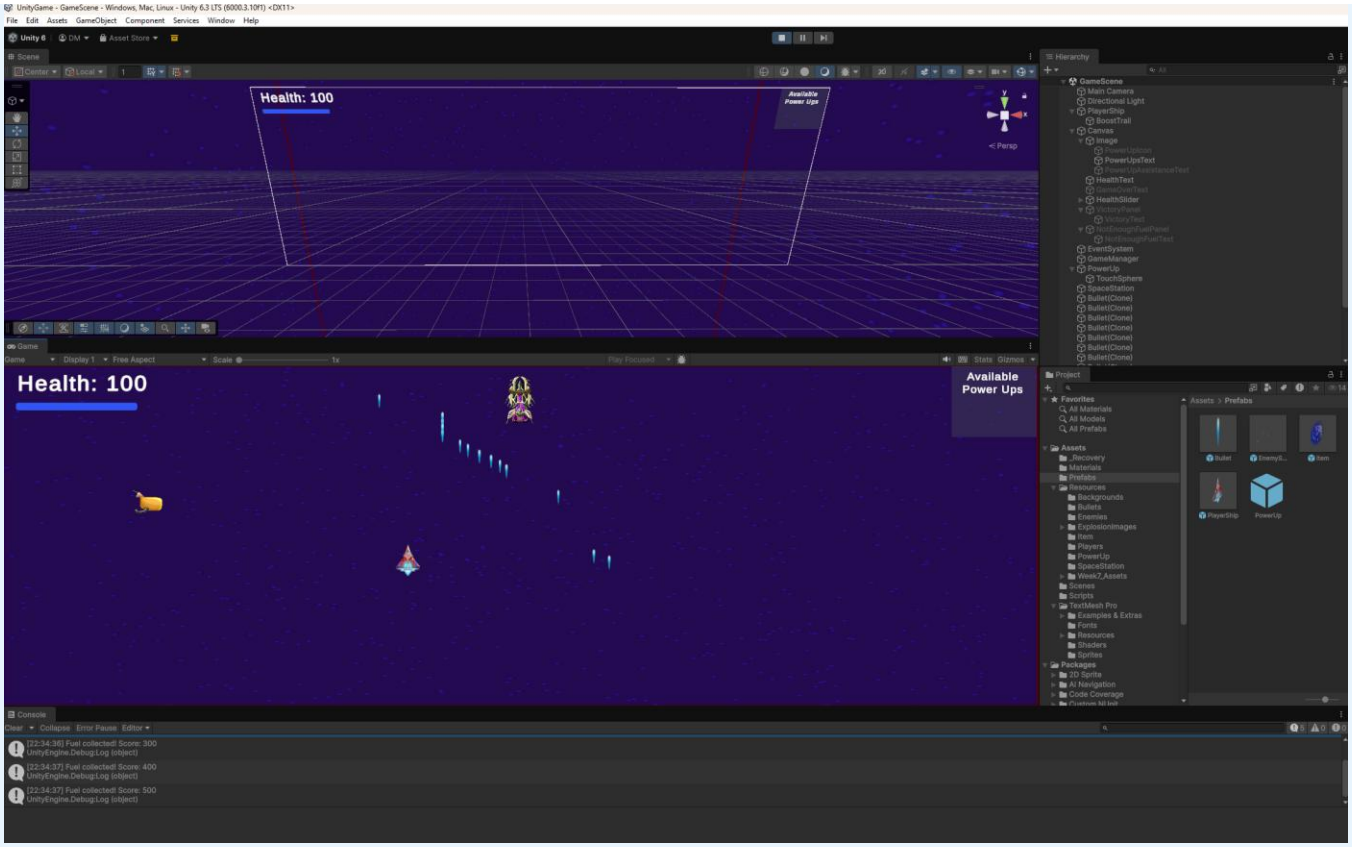


Please see the inline comments in PlayerShip.cs with ActivateBoost(), FlashShip(), and OnCollisionStay() which are all fully commented in the project files.

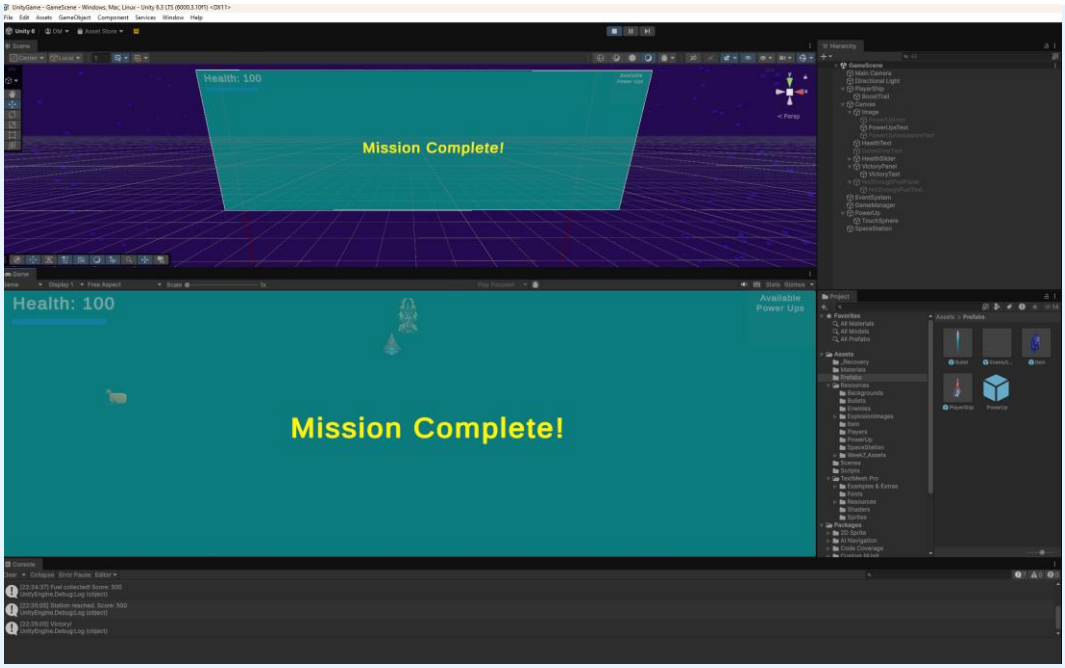
Feature 3: Victory Condition - Feature 1: Creative Win State

DESCRIPTION

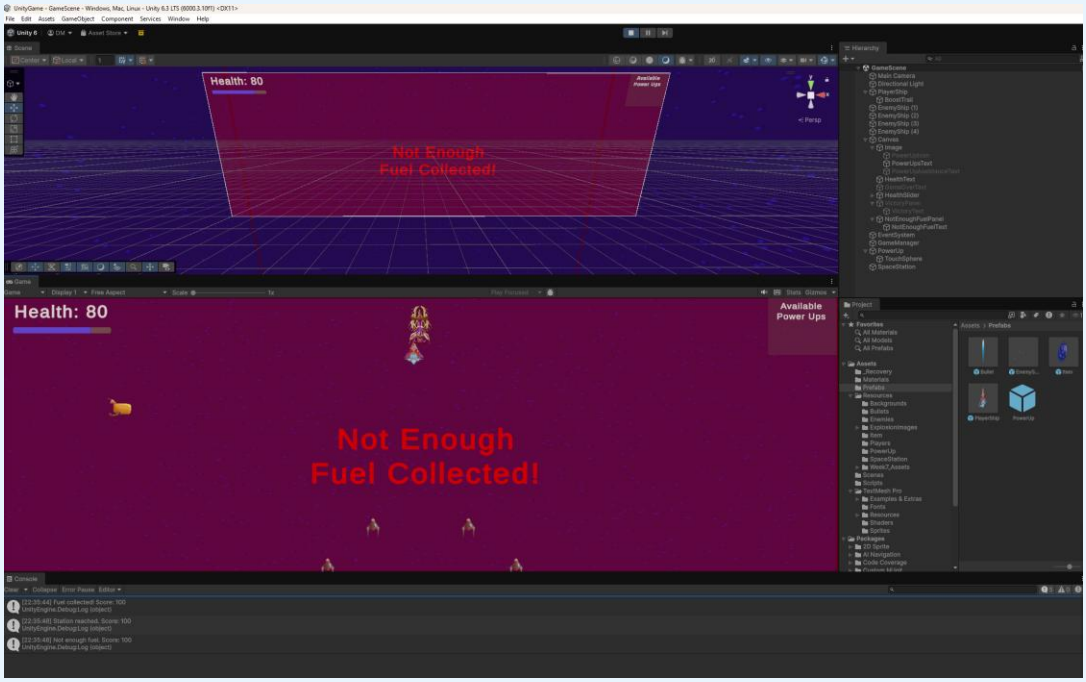
The player wins the game by successfully navigating their rocket ship to a 'Space Station' target object positioned at the top of the level, while accumulating a minimum score of 500 points (earned by destroying enemies). Both conditions must be met simultaneously - reaching the station with fewer than 500 points triggers a 'Not enough fuel collected' failure message instead. On victory, a 'Mission Complete' screen is displayed meaning the player has collected enough fuel points to complete the game.



This screenshot shows the game running in the Unity Editor with the Console window open at the bottom. As the player destroys enemy ships during gameplay, the Console displays a series of "Fuel collected! Score: X" messages, confirming that the score system is working correctly. Each enemy destroyed increments the score by 10 points, simulating fuel being collected from defeated enemies. This provides real-time verification that the AddScore() method in GameManager.cs is being called successfully each time an enemy is destroyed.



This screenshot shows the Victory state being triggered. After the player accumulated 500 or more points by destroying enemies and then flew their ship up to the Space Station, the "Mission Complete!" panel activates and fills the screen. The game is simultaneously frozen using Time.timeScale = 0 where enemies stop spawning, signalling a clean end to the gameplay loop. This confirms that both win conditions (score ≥ 500 and reaching the station) were met simultaneously.



This screenshot shows the failure state being triggered. The player reached the Space Station before accumulating 500 points, causing the "Not Enough Fuel Collected!" panel to appear in red. The player's health reads 80, meaning they took some damage during gameplay but did not fight enough enemies to meet the fuel requirement. After 3 seconds the panel automatically disappears, allowing the player to fly back down and continue collecting fuel, keeping the game going rather than forcing a restart.

REASONING

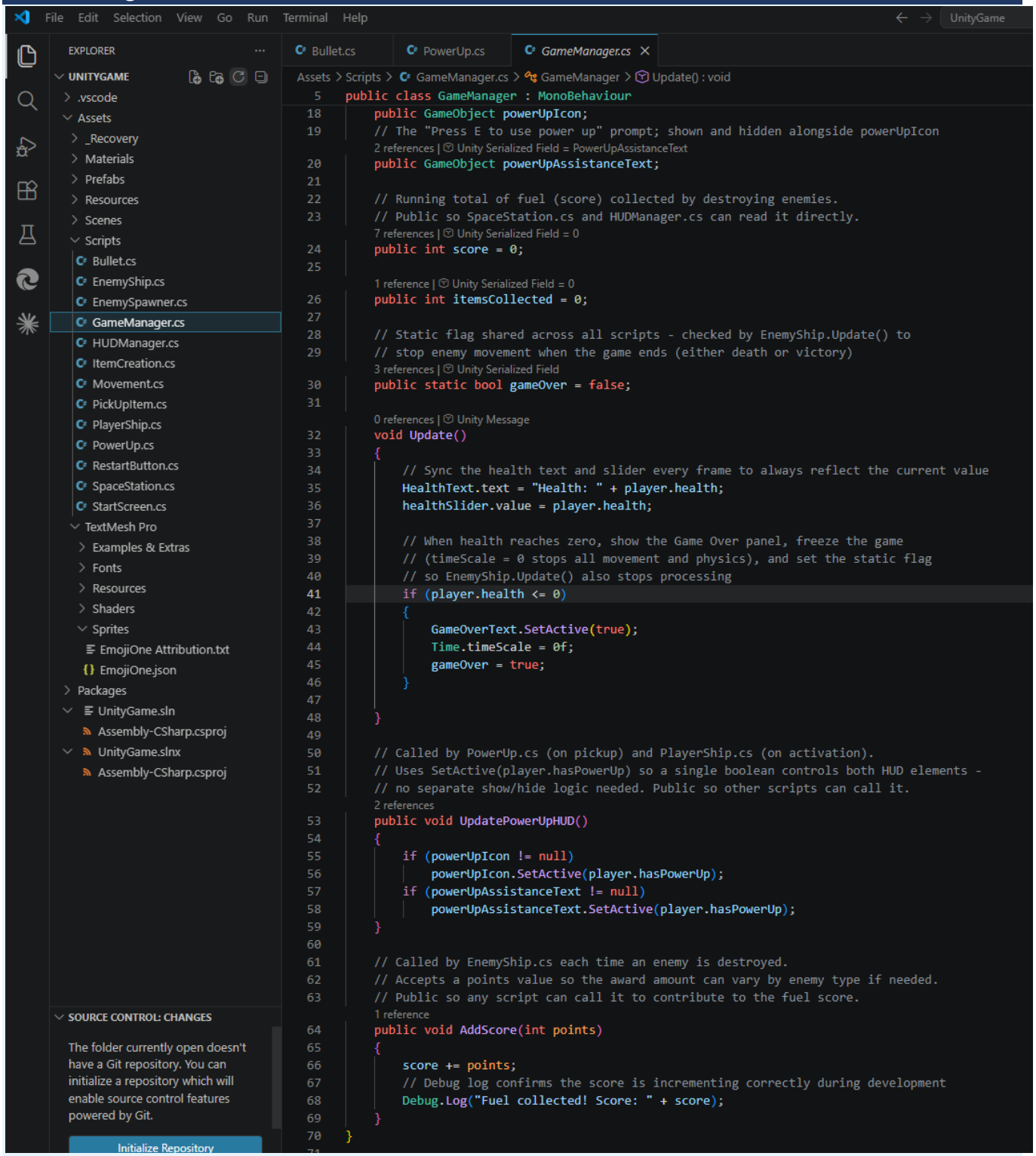
A win condition requiring two simultaneous conditions (position + score) is more interesting and challenging than a simple 'destroy all enemies' win. It forces the player to balance exploration/navigation against combat - they need to fight enough enemies to build score but also progress towards the station. This dual-condition design creates natural tension and meaningful decision-making. The thematic framing (fuel/score = preparation for the journey) keeps the win state coherent with the rocket ship setting.

A potential future improvement would be to add a countdown timer that constrains how long the player has to collect the 500 points required to win. This would introduce a third source of pressure on top of position and score, transforming the current "balance two goals" design into a more dynamic "balance three goals under time pressure" scenario. It would also reduce the viability of conservative play styles, encouraging more aggressive engagement with enemies and adding replay variety as players try to optimise their route.

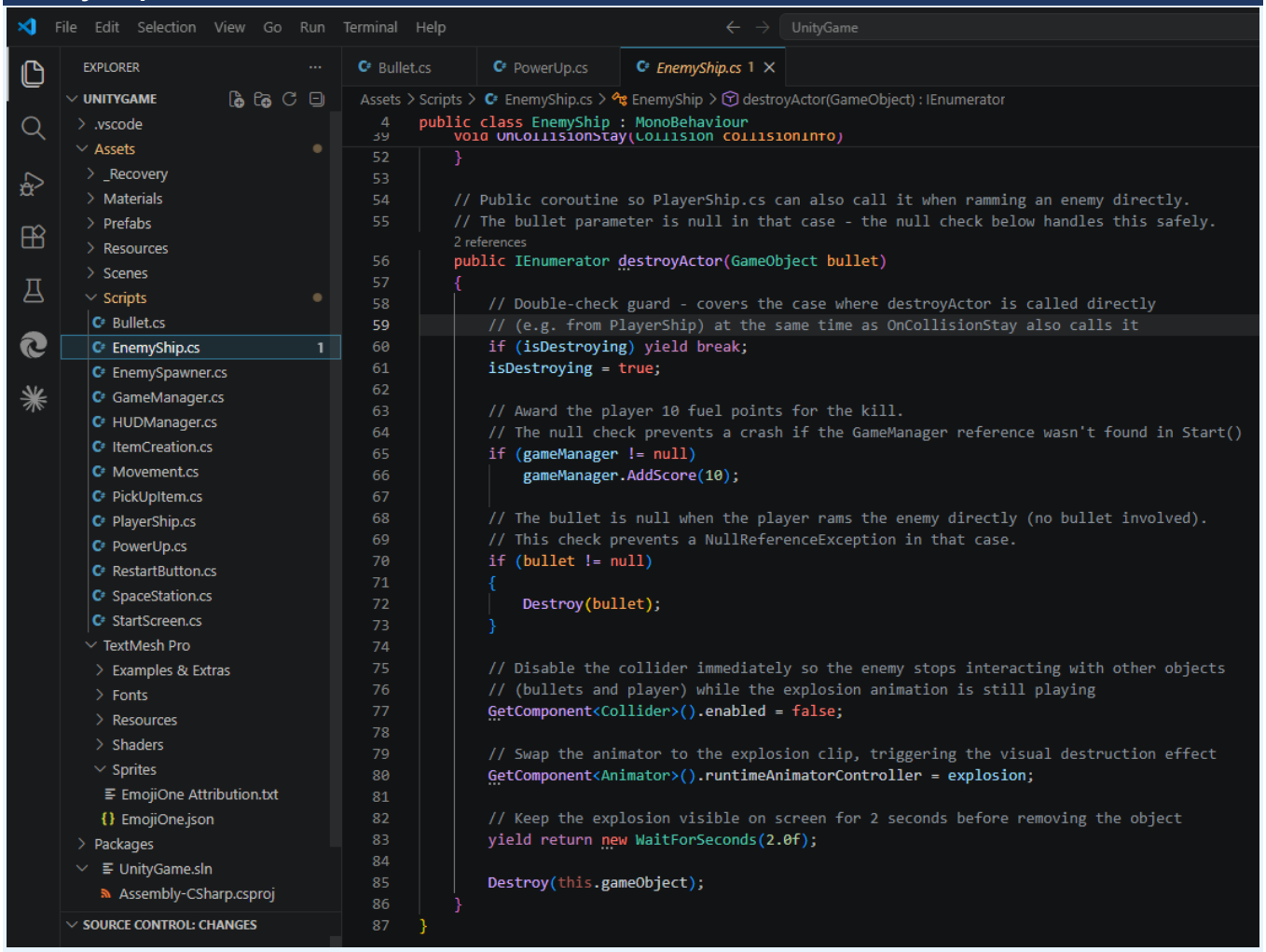
BLUEPRINTS/SCRIPTS AND IMPLEMENTATION

The win condition is implemented in **SpaceStation.cs**, which lives on the Space Station GameObject. Its OnTriggerEnter(Collider other) method fires when any collider enters the station's trigger zone. A CompareTag("Player") filter ensures only the player ship can activate the check - bullets and enemies are ignored. Inside the player branch, the method reads gameManager.score and evaluates the dual win condition: if the score is **≥ 500**, the victoryPanel GameObject is activated (showing the "Mission Complete!" UI) and Time.timeScale = 0f freezes the game. If the score is below 500, a coroutine ShowNotEnoughFuel() is launched instead. It activates the notEnoughFuelPanel, waits 3 seconds via WaitForSeconds(3f), then hides the panel automatically so the player can continue collecting fuel without needing to restart. The "Play Again" / restart flow is handled separately by RestartButton.cs (see Extras section).

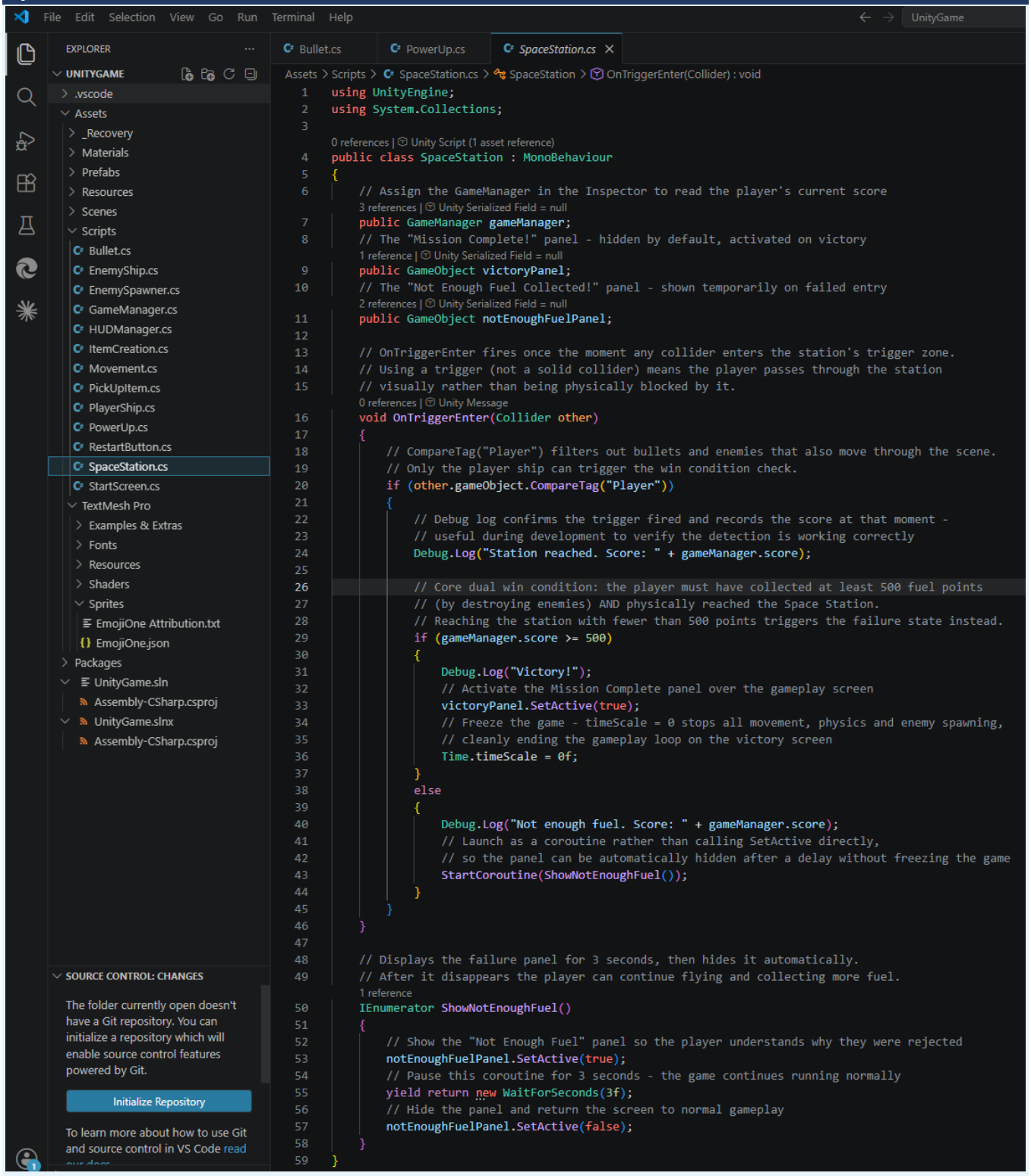
GameManager.cs



EnemyShip.cs



SpaceStation.cs

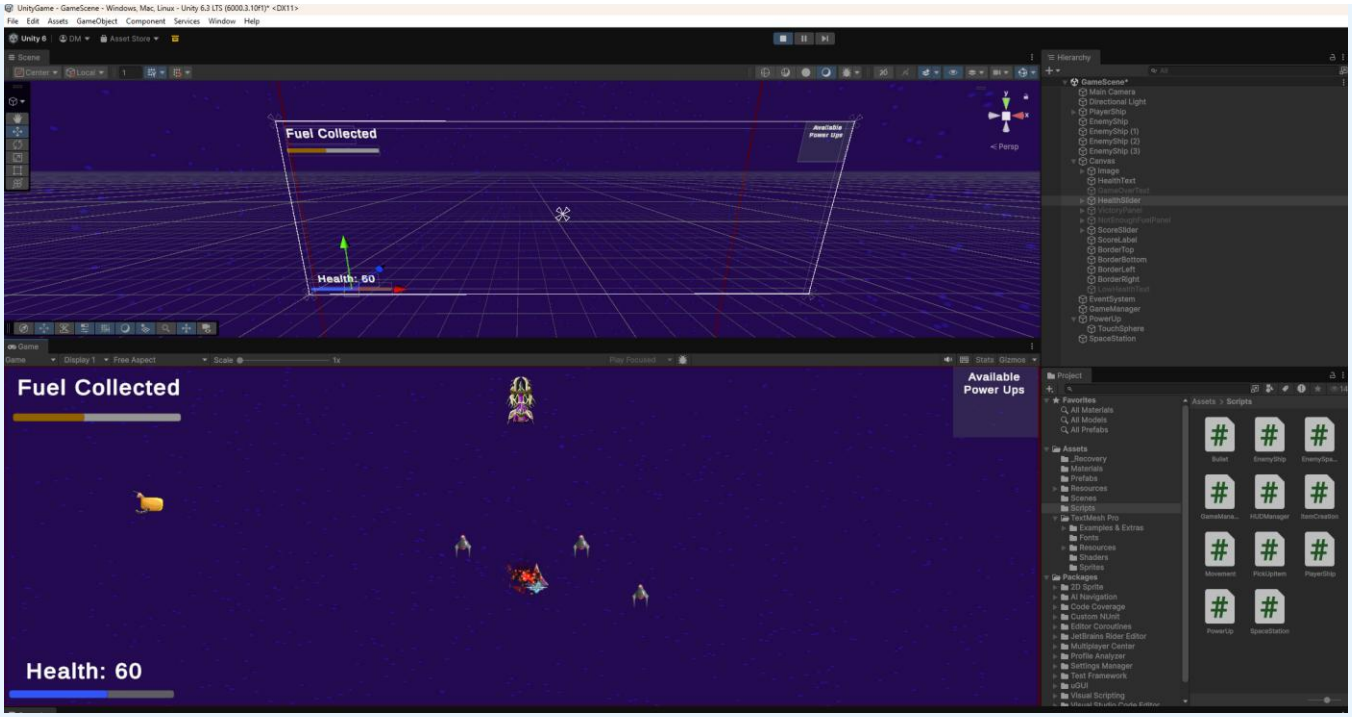


Please see the inline comments in SpaceStation.cs, GameManager.cs and EnemyShip.cs for an explanation of the dual win condition check and score award logic.

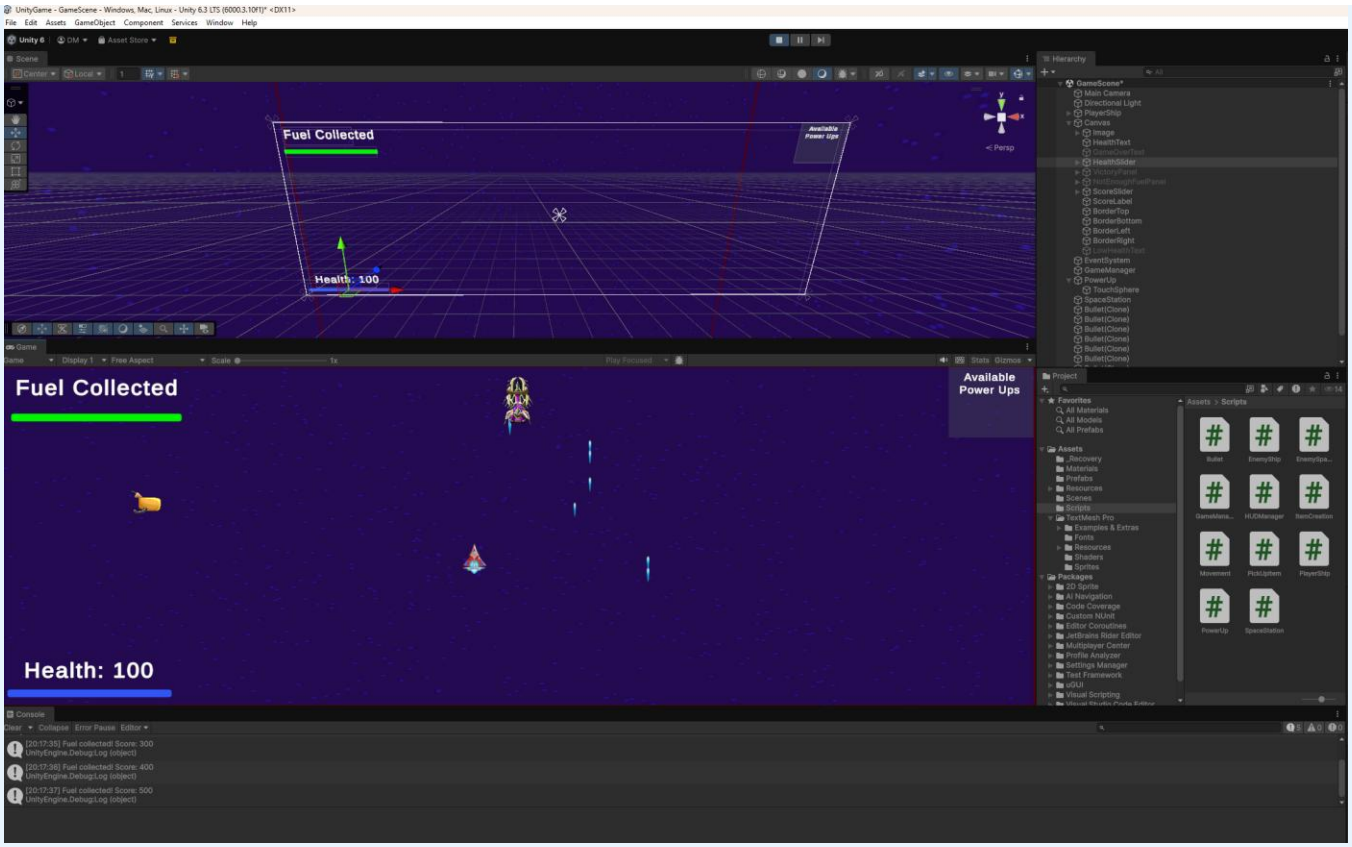
Feature 4: Victory Condition - Feature 2: Real-Time Victory Progress Feedback

DESCRIPTION

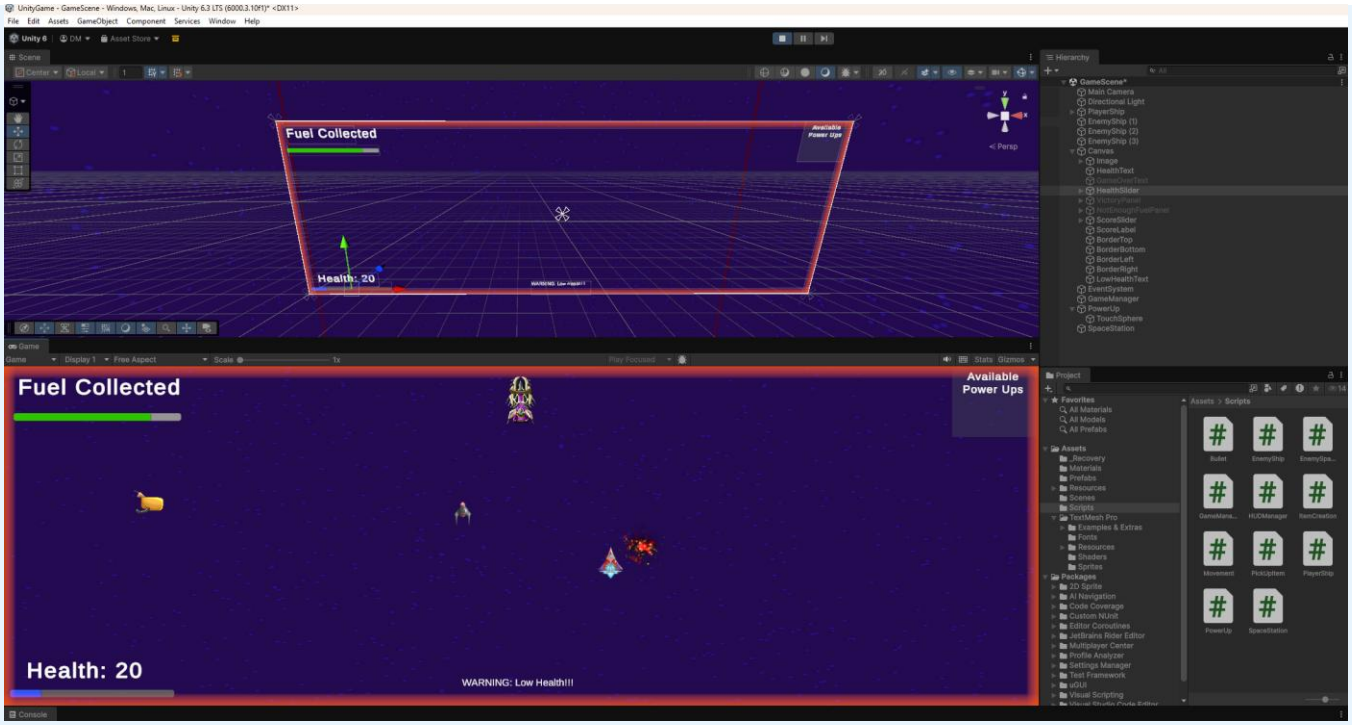
Two real-time HUD elements provide feedback on the player's progress during gameplay: (1) a fuel progress bar: A UI Slider that fills as the player's score approaches 500, with the fill colour changing from red to green as it fills, giving the player an immediate visual indicator of how close they are to the win condition. (2) a low health warning; A red flashing border that appears around the edges of the screen with a warning message when the player's health drops to 30 or below, alerting them that they are in danger. Both elements are always active during gameplay.



This screenshot shows the fuel progress bar early in gameplay. The slider is partially filled with an orange colour, indicating the player has collected some fuel but has not yet reached the 500-point target. The colour at this stage sits between red and green as a result of the Color.Lerp function visually communicating to the player that they are making progress but still have more enemies to destroy before they can complete the mission.



This screenshot shows the fuel progress bar fully filled and displaying green, confirming the player has reached 500 points. The slider value matches the score exactly, as it is updated every frame in the HUDManager Update() method. The green colour signals to the player that both win conditions are now achievable. They have enough fuel and can fly to the Space Station to complete the mission.



This screenshot shows the low health warning system activating. With the player's health at 20 (below the 30-point threshold) the red border panels appear around the edges of the screen and pulse in and out using a sine wave calculation. The "WARNING: Low Health!" text is simultaneously made visible at the bottom of the screen. Together these two elements create an urgent visual signal that the player is close to death, encouraging them to avoid enemies or use their boost power-up to survive.

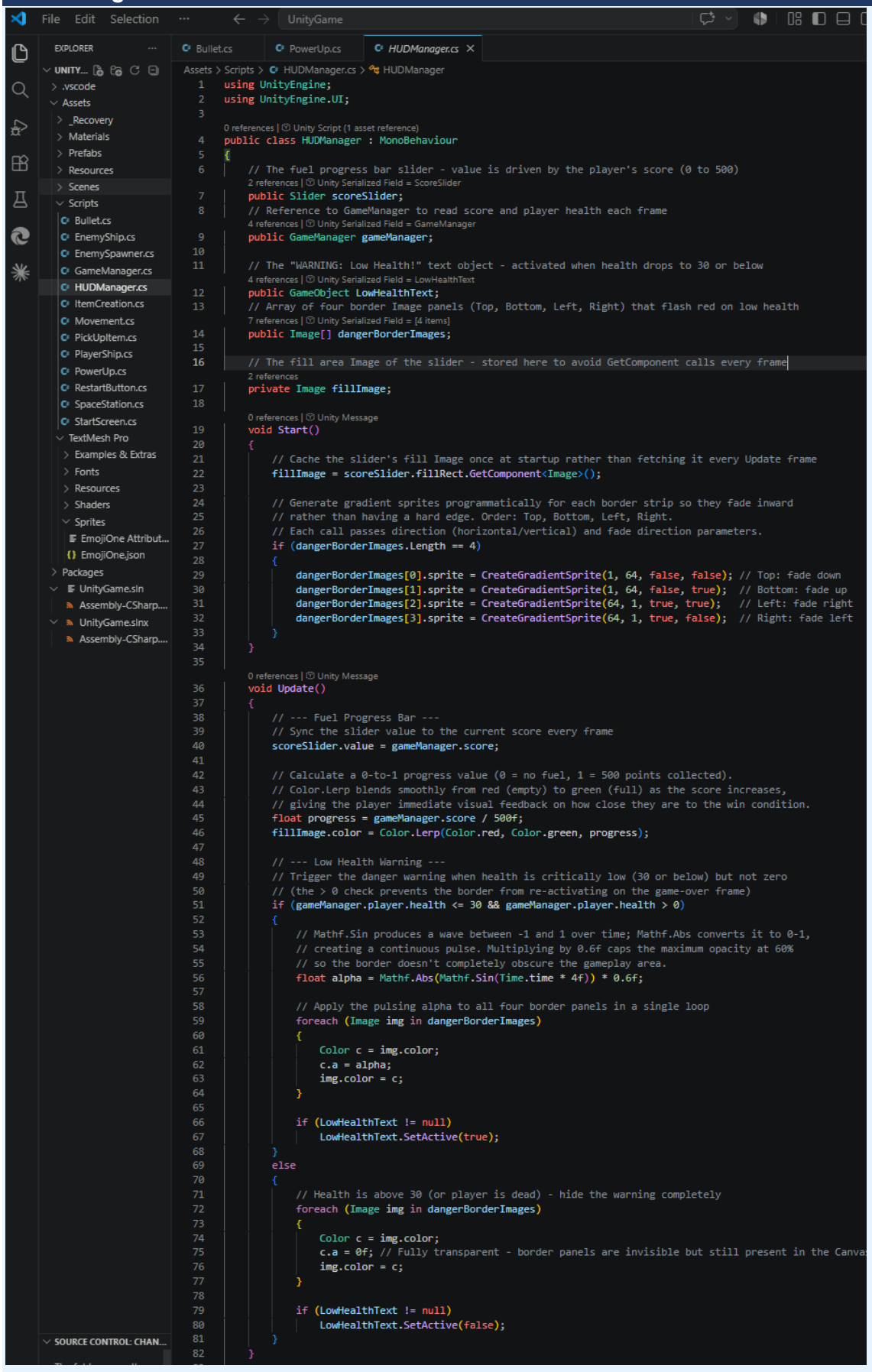
REASONING

Real-time feedback is essential for a good player experience and without it, the player has no way to gauge how close they are to winning or how much danger they are in. The fuel progress bar was chosen because the colour shift from red to green provides immediate, intuitive feedback without the player needing to read any numbers. A full green bar signals readiness to complete the mission at a glance. The low health warning border was chosen because a flashing edge effect is a well-established game design pattern for communicating danger, drawing the player's attention without blocking the gameplay area. Together the two elements cover both win progress and survival status simultaneously.

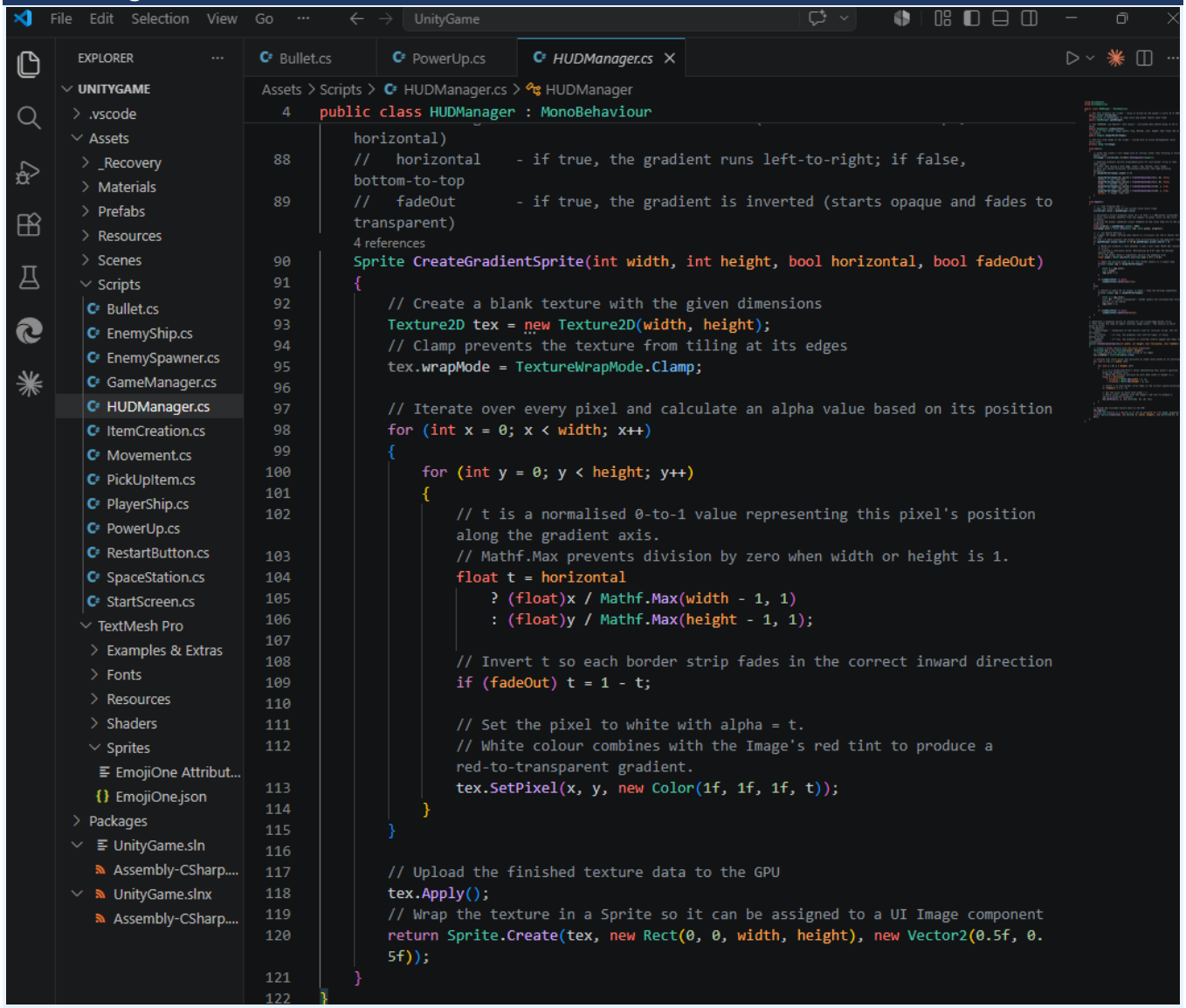
BLUEPRINTS/SCRIPTS AND IMPLEMENTATION

The HUDManager script runs in Update() and controls both HUD elements every frame. For the fuel progress bar, scoreSlider.value is set directly to gameManager.score, and the fill area's Image component colour is lerped between Color.red and Color.green using Color.Lerp(Color.red, Color.green, gameManager.score / 500f), producing a smooth colour transition as the score increases. For the low health warning, the script checks whether gameManager.player.health is at or below 30. If true, the four border Image components have their alpha driven by Mathf.Abs(Mathf.Sin(Time.time * 4f)) * 0.6f, creating a continuous pulsing flash effect, and the LowHealthText GameObject is activated. The * 0.6f caps max opacity at 60% so the border doesn't fully obscure gameplay. When health rises back above 30 the border alpha is set to 0 and the text is hidden. Gradient sprites are generated programmatically in Start() using Texture2D so each border fades smoothly inward rather than having a hard edge.

HUDManager.cs



HUDManager.cs



GameManager.cs

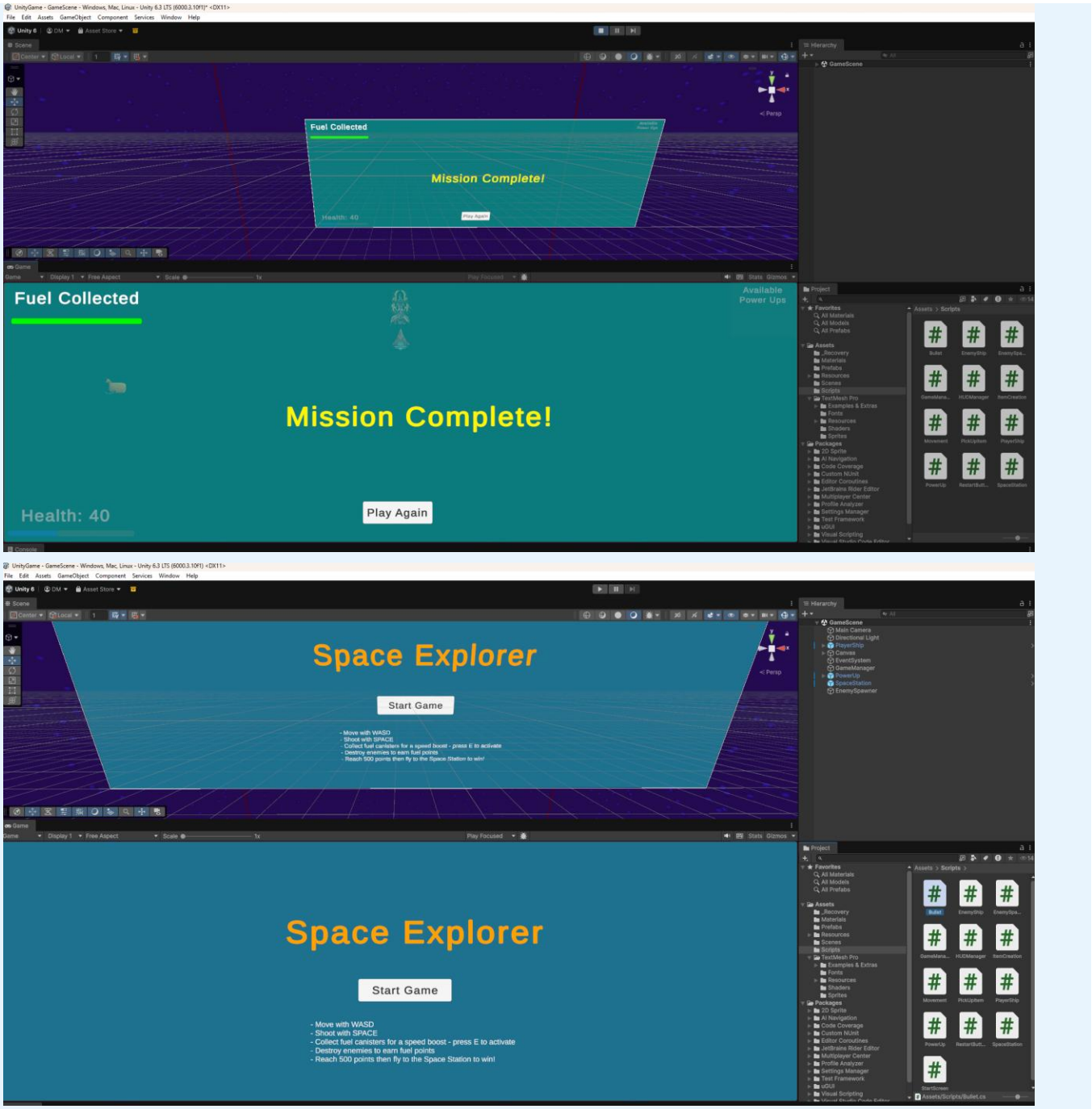

```
using UnityEngine;
using TMPro;
using UnityEngine.UI;

7 references | Unity Script (1 asset reference)
public class GameManager : MonoBehaviour
{
    // Reference to the player ship - used to read health and hasPowerUp each frame
    7 references | Unity Serialized Field = PlayerShip
    public PlayerShip player;
    // TextMeshPro text element that displays the current health value on the HUD
    1 reference | Unity Serialized Field = HealthText
    public TextMeshProUGUI HealthText;
    // The Game Over panel, hidden by default and activated when the player dies
    1 reference | Unity Serialized Field = GameOverText
    public GameObject GameOverText;

    // UI Slider that visually represents the player's current health (max 100)
    1 reference | Unity Serialized Field = HealthSlider
    public Slider healthSlider;

    // The canister icon shown on the HUD when a power-up is held; hidden when it is not
    2 references | Unity Serialized Field = PowerUpIcon
    public GameObject powerUpIcon;
    // The "Press E to use power up" prompt; shown and hidden alongside powerUpIcon
    2 references | Unity Serialized Field = PowerUpAssistanceText
    public GameObject powerUpAssistanceText;

    // Running total of fuel (score) collected by destroying enemies.
    // Public so SpaceStation.cs and HUDManager.cs can read it directly.
    7 references | Unity Serialized Field = 0
    public int score = 0;

    1 reference | Unity Serialized Field = 0
    public int itemsCollected = 0;

    // Static flag shared across all scripts - checked by EnemyShip.Update() to
    // stop enemy movement when the game ends (either death or victory)
    3 references | Unity Serialized Field
    public static bool gameOver = false;

    0 references | Unity Message
    void Update()
    {
        // Sync the health text and slider every frame to always reflect the current
        value
        HealthText.text = "Health: " + player.health;
        healthSlider.value = player.health;

        // When health reaches zero, show the Game Over panel, freeze the game
        // (timeScale = 0 stops all movement and physics), and set the static flag
        // so EnemyShip.Update() also stops processing
        if (player.health <= 0)
        {
            GameOverText.SetActive(true);
            Time.timeScale = 0f;
            gameOver = true;
        }

    }

    // Called by PowerUp.cs (on pickup) and PlayerShip.cs (on activation).
    // Uses SetActive(player.hasPowerUp) so a single boolean controls both HUD elements
    -
    // no separate show/hide logic needed. Public so other scripts can call it.
    2 references
    public void UpdatePowerUpHUD()
    {
        if (powerUpIcon != null)
            powerUpIcon.SetActive(player.hasPowerUp);
        if (powerUpAssistanceText != null)
            powerUpAssistanceText.SetActive(player.hasPowerUp);
    }

    // Called by EnemyShip.cs each time an enemy is destroyed.
    // Accepts a points value so the award amount can vary by enemy type if needed.
    // Public so any script can call it to contribute to the fuel score.
    1 reference
    public void AddScore(int points)
    {
        score += points;
        // Debug log confirms the score is incrementing correctly during development
        Debug.Log("Fuel collected! Score: " + score);
    }
}
```

Please see the inline comments in HUDManager.cs and GameManager.cs for an explanation of the Color.Lerp fuel bar, the sine-wave pulse effect, and the programmatic gradient sprites

Extra Transformations or Features

DESCRIPTION

A main menu start screen, and a full restart system were added to provide a complete game loop. When the scene first loads, the game is immediately frozen, and a start panel is displayed over the gameplay area. The player must press the Start button to begin, at which point the game resumes and the panel is hidden. On game over or victory, a Restart button is available which reloads the scene and returns the player to the start screen, ready to play again.



This screenshot shows the Restart button being pressed after a game over. The scene reloads and the start panel reappears over the gameplay area, returning the player to the beginning of the game loop.

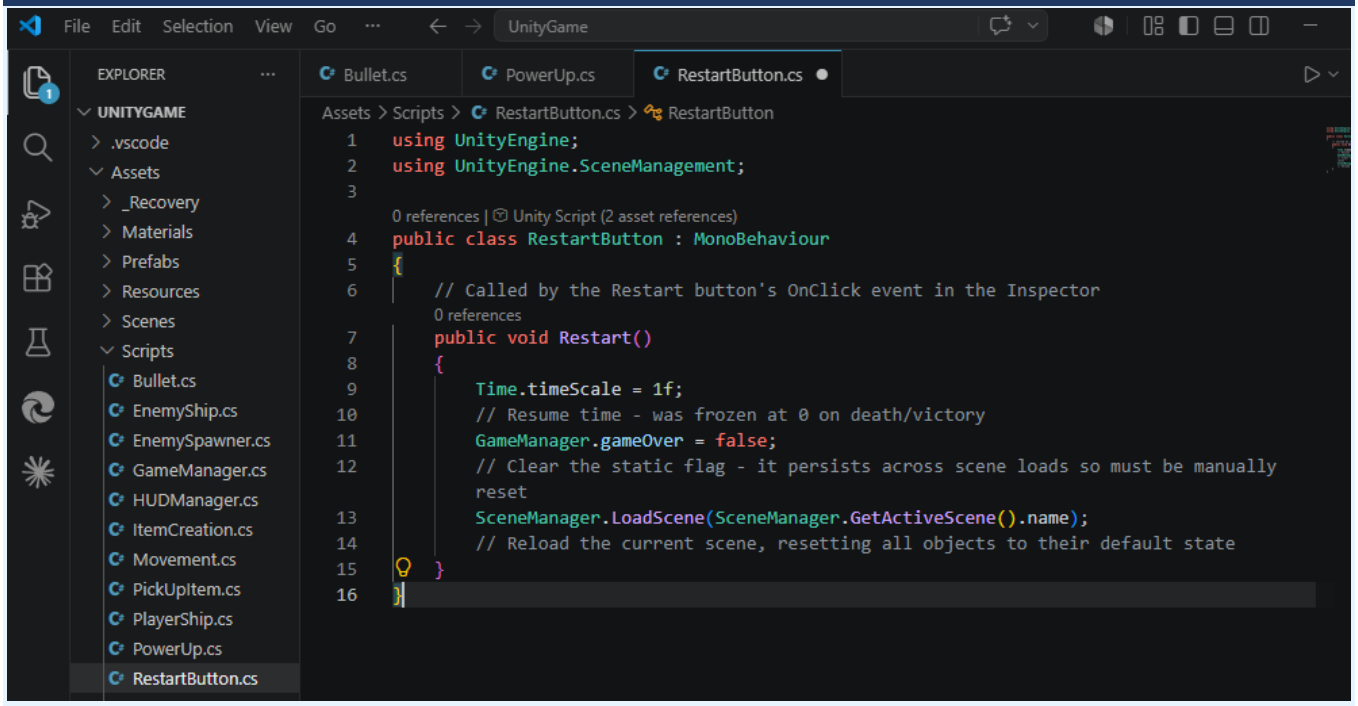
REASONING

A start screen was added because launching directly into gameplay without any menu is a poor user experience. The player has no time to prepare and enemies may already be moving before the game has started. Freezing time with Time.timeScale = 0 on scene load ensures nothing happens until the player deliberately presses Start, giving them full control over when the game begins. The restart system was added to close the game loop. Without it, the player would need to manually stop and restart the game in the editor after dying or winning, which is not acceptable in a finished product. Reloading the scene is the simplest and most reliable way to reset all game state at once, and returning to the start screen rather than jumping straight back into gameplay keeps the flow consistent with the first launch.

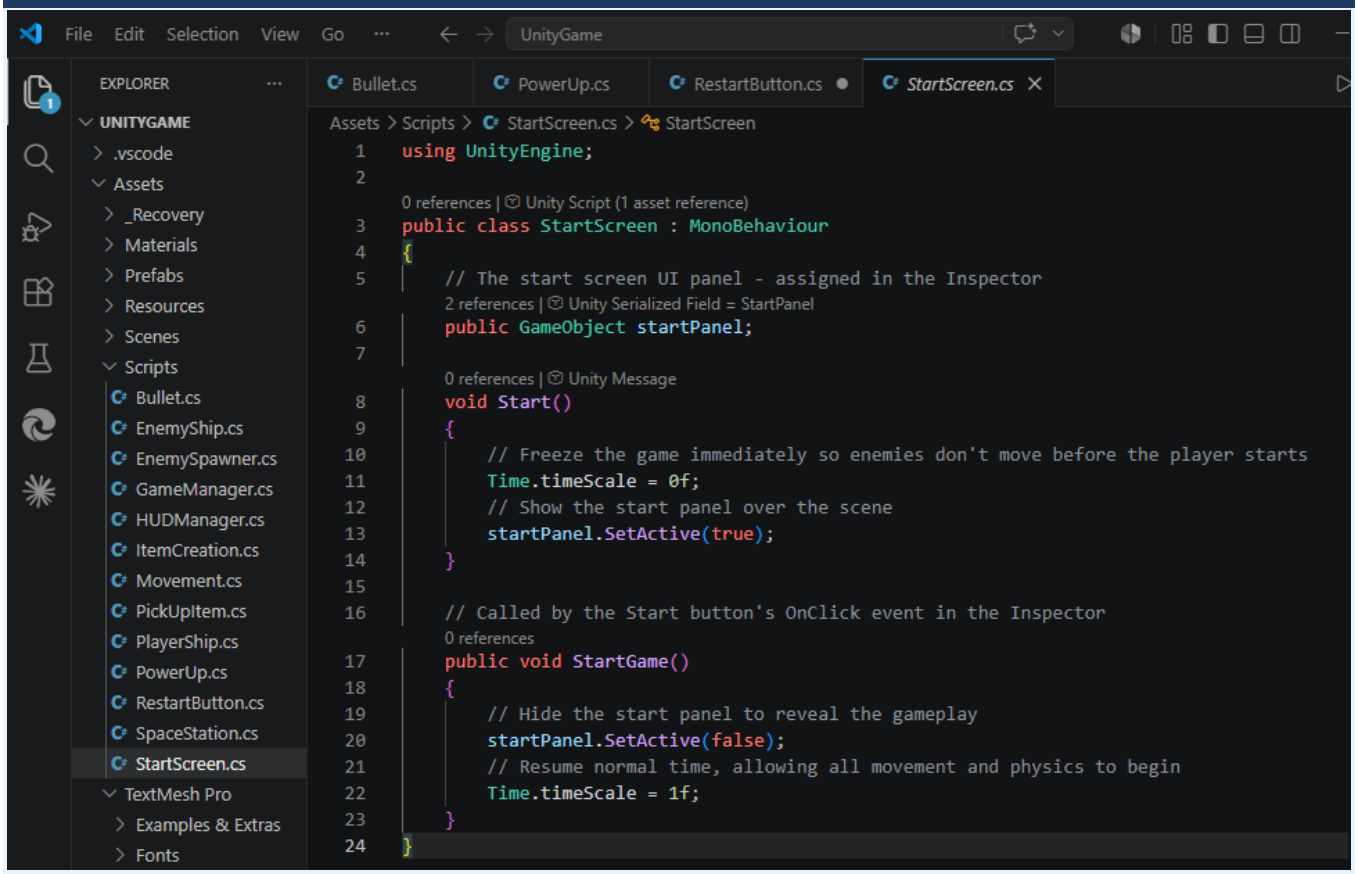
BLUEPRINTS/SCRIPTS AND IMPLEMENTATION

The start screen is controlled by StartScreen.cs. In Start(), Time.timeScale is set to 0f and the start panel GameObject is activated. This freezes all physics and movement the moment the scene loads. A public StartGame() method is wired to the Start button's OnClick event in the Inspector. When called, it hides the panel with SetActive(false) and restores Time.timeScale to 1f, resuming normal gameplay. The restart system is controlled by RestartButton.cs. Its public Restart() method, wired to the Restart button's OnClick event, performs three steps: Time.timeScale is restored to 1f (since it was frozen at 0 on game over or victory), the static GameManager.gameOver flag is reset to false (this flag persists across scene loads and must be manually cleared, otherwise EnemyShip.Update() would remain frozen after the reload), and SceneManager.LoadScene(SceneManager.GetActiveScene().name) reloads the current scene by name, destroying all existing objects and re-running Start() on every script including StartScreen, which shows the start panel again.

RestarButton.cs



StartScreen.cs



See the inline comments in StartScreen.cs and RestartButton.cs for a full explanation of the menu and restart logic.

OTHER ADDITIONS

Several smaller additions were made to the game beyond the four core features that are worth noting. Dynamic spawn difficulty was added via EnemySpawner.cs: the spawnInterval is reduced by 0.05 seconds every time an enemy spawns, causing enemies to appear more frequently as the game progresses. The interval is capped at a minimum of 0.5 seconds using Mathf.Max to prevent the game from becoming unplayable. Because InvokeRepeating cannot change its own interval once started, the schedule is cancelled with CancelInvoke and restarted with the new interval after every spawn. A screen boundary system was added to PlayerShip.cs using Camera.main.ViewportToWorldPoint to convert the screen edges into world-space coordinates, which are then used as min and max values for Mathf.Clamp on both axes, preventing the player from flying off-screen entirely. A health slider was also added to the HUD in GameManager.cs alongside the existing health text, giving a visual bar representation of remaining health that is updated every frame to match the player's current health. The sprites for the power up and the space station were also updated to give the game a more polished look.

References

Anthropic Claude (claude.ai) was used during this project to assist with debugging, understanding unfamiliar code concepts, navigating Unity-specific implementation, and working through implementation challenges that went beyond what was covered in class. This included setting up multiple coroutines running simultaneously (ActivateBoost and FlashShip), scripting particle system control, creating dynamic HUD elements using Color.Lerp for colour transitions and Mathf.Sin for pulsing animations, generating gradient sprites programmatically with Texture2D.SetPixel and Sprite.Create, dynamically adjusting spawn intervals using CancelInvoke and InvokeRepeating, clamping the player to screen boundaries using Camera.main.ViewportToWorldPoint, and correctly managing a static flag reset across scene reloads. All final code was written, tested, and implemented by me.

Unity Technologies (n.d.) Layer-based collision detection, Unity Technologies, accessed 29 April 2026. https://docs.unity3d.com/Manual/LayerBasedCollision.html

Unity Technologies (n.d.) Time.timeScale, Unity Technologies, accessed 1 May 2026. https://docs.unity3d.com/ScriptReference/Time-timeScale.html

Unity Technologies (n.d.) Sprite.Create, Unity Technologies, accessed 3 May 2026. https://docs.unity3d.com/ScriptReference/Sprite.Create.html

Unity Technologies (n.d.) Texture2D.SetPixel, Unity Technologies, accessed 7 May 2026. https://docs.unity3d.com/ScriptReference/Texture2D.SetPixel.html

Unity Technologies (n.d.) Color.Lerp, Unity Technologies, accessed 9 May 2026. https://docs.unity3d.com/ScriptReference/Color.Lerp.html

Unity Technologies (n.d.) Mathf.Sin, Unity Technologies, accessed 10 May 2026. https://docs.unity3d.com/ScriptReference/Mathf.Sin.html

Unity Technologies (n.d.) Coroutines, Unity Technologies, accessed 10 May 2026. https://docs.unity3d.com/Manual/Coroutines.html

Unity Technologies (n.d.) MonoBehaviour.InvokeRepeating, Unity Technologies, accessed 10 May 2026. https://docs.unity3d.com/ScriptReference/MonoBehaviour.InvokeRepeating.html

CraftPix (n.d.) Free Cyberpunk Energy Items 512x512 Icon Pack, CraftPix, accessed 16 May 2026. https://craftpix.net/freebies/free-cyberpunk-energy-items-512x512-icon-pack/